

Data Quality Verification

SQL Queries : Accuracy, Completeness & Reliability

Note : For this created a temporary tables in <https://sqliteonline.com/> to perform DQ checks.

Source Table	File Format	Rows
dim_customer	Excel (.xls)	250
fact_order	CSV	250
fact_shipping	JSON Array	250

Table of Contents

1. Completeness Checks

1.1 Customer , fact_order ,fact_shipping - **NULL Checks & Row Count**

2. Accuracy Checks : Customer

2.1 Duplicate Customer_ID

2.2 Invalid characters in name

3. Accuracy Checks : fact_order

3.1 Duplicate Order_ID

3.2 Negative or zero Amount

3.3 Amount statistics

3.4 Customers with multiple orders

4. Accuracy Checks : fact_shipping

- 4.1 Duplicate Shipping_ID
- 4.2 Customers with multiple shipments

5. Referential Integrity Checks8

- 5.1 Orphan orders
- 5.2 Orphan shipments
- 5.3 Customers Engagement details in Order and Shipping data

1. Completeness Checks : NULL & Row Count

Every column across all three tables is checked for NULL values. A count of zero confirms the column is fully populated.

1.1 dim_customer : NULL check per column

We use COUNT(*) with a FILTER clause for each column to count NULL values ,
Result of count of Null value is zero across all columns indicating data is Complete.

The screenshot shows a database client interface with a sidebar on the left containing a tree view of databases and tables. The main area displays a SQL query and its results. The query is a SELECT statement that counts the total rows and the number of NULL values for each column in the dim_customer table. The results are shown in a table with 6 columns: total_rows, null_customer_id, null_first_name, null_last_name, null_age, and null_country. The values are 250, 0, 0, 0, 0, and 0 respectively.

```
1 SELECT
2   COUNT(*)                                AS total_rows,
3   COUNT(*) FILTER (WHERE Customer_ID IS NULL) AS null_customer_id,
4   COUNT(*) FILTER (WHERE First IS NULL)      AS null_first_name,
5   COUNT(*) FILTER (WHERE Last IS NULL)       AS null_last_name,
6   COUNT(*) FILTER (WHERE Age IS NULL)        AS null_age,
7   COUNT(*) FILTER (WHERE Country IS NULL)    AS null_country
8 FROM dim_customer;
9
```

total_rows	null_customer_id	null_first_name	null_last_name	null_age	null_country
250	0	0	0	0	0

```
SELECT COUNT(*) AS total_rows ,COUNT(*) FILTER (WHERE Customer_ID IS NULL) AS null_customer_id ,COUNT(*) FILTER (WHERE First IS NULL) AS null_first_name,
COUNT(*) FILTER (WHERE Last IS NULL) AS null_last_name, COUNT(*) FILTER (WHERE Age IS NULL) AS null_age,COUNT(*) FILTER (WHERE Country IS NULL) AS
null_country FROM dim_customer;
```

1.2 fact_order : NULL check per column

We use COUNT(*) with a FILTER clause for each column to count NULL values ,
Result of count of Null value is zero across all columns indicating data is Complete.

The screenshot shows a database client interface with a sidebar on the left and a main query editor on the right. The sidebar lists the database 'Demo.Memory' and its schema, including tables 'demo' and 'dim_customer', and columns like 'customer_id', 'first', 'last', 'age', and 'country'. The main editor displays a SQL query that counts the total rows and the number of NULL values for each column in the 'fact_order' table. The query is as follows:

```
1 SELECT
2   COUNT(*)
3   COUNT(*) FILTER (WHERE Order_ID IS NULL)
4   COUNT(*) FILTER (WHERE Item IS NULL)
5   COUNT(*) FILTER (WHERE Amount IS NULL)
6   COUNT(*) FILTER (WHERE Customer_ID IS NULL)
7 FROM fact_order;
```

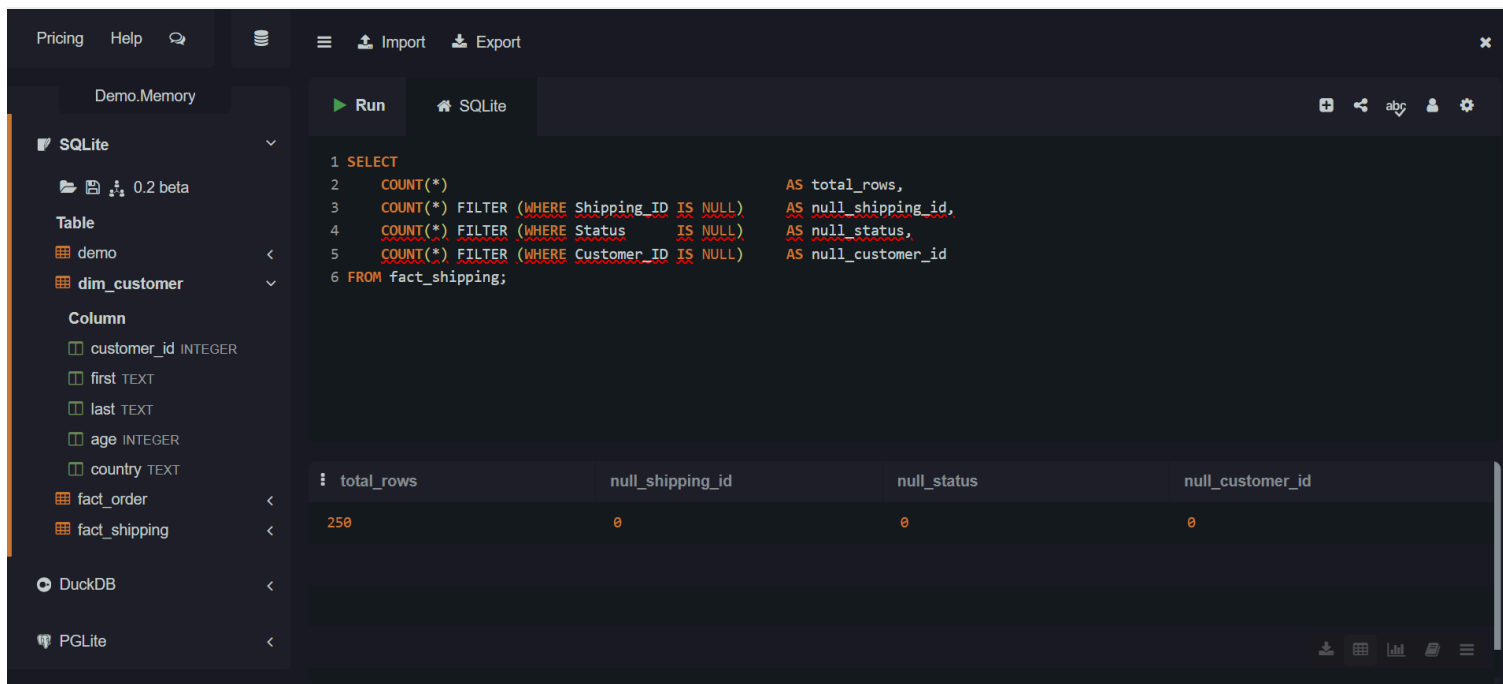
The results of the query are displayed in a table below the query editor:

total_rows	null_order_id	null_item	null_amount	null_customer_id
250	0	0	0	0

Query : `SELECT COUNT(*) AS total_rows, COUNT(*) FILTER (WHERE Order_ID IS NULL) AS null_order_id, COUNT(*) FILTER (WHERE Item IS NULL) AS null_item, COUNT(*) FILTER (WHERE Amount IS NULL) AS null_amount, COUNT(*) FILTER (WHERE Customer_ID IS NULL) AS null_customer_id FROM fact_order;`

1.3 fact_shipping : NULL check per column

We use COUNT(*) with a FILTER clause for each column to count NULL values ,
Result :count of Null value is zero across all columns indicating data is Complete.



The screenshot shows a database client interface with a sidebar on the left containing a tree view of the database structure. The main area displays a SQL query and its results. The query is as follows:

```
1 SELECT
2   COUNT(*)
3   COUNT(*) FILTER (WHERE Shipping_ID IS NULL)
4   COUNT(*) FILTER (WHERE Status IS NULL)
5   COUNT(*) FILTER (WHERE Customer_ID IS NULL)
6 FROM fact_shipping;
```

The results are displayed in a table with the following columns and values:

total_rows	null_shipping_id	null_status	null_customer_id
250	0	0	0

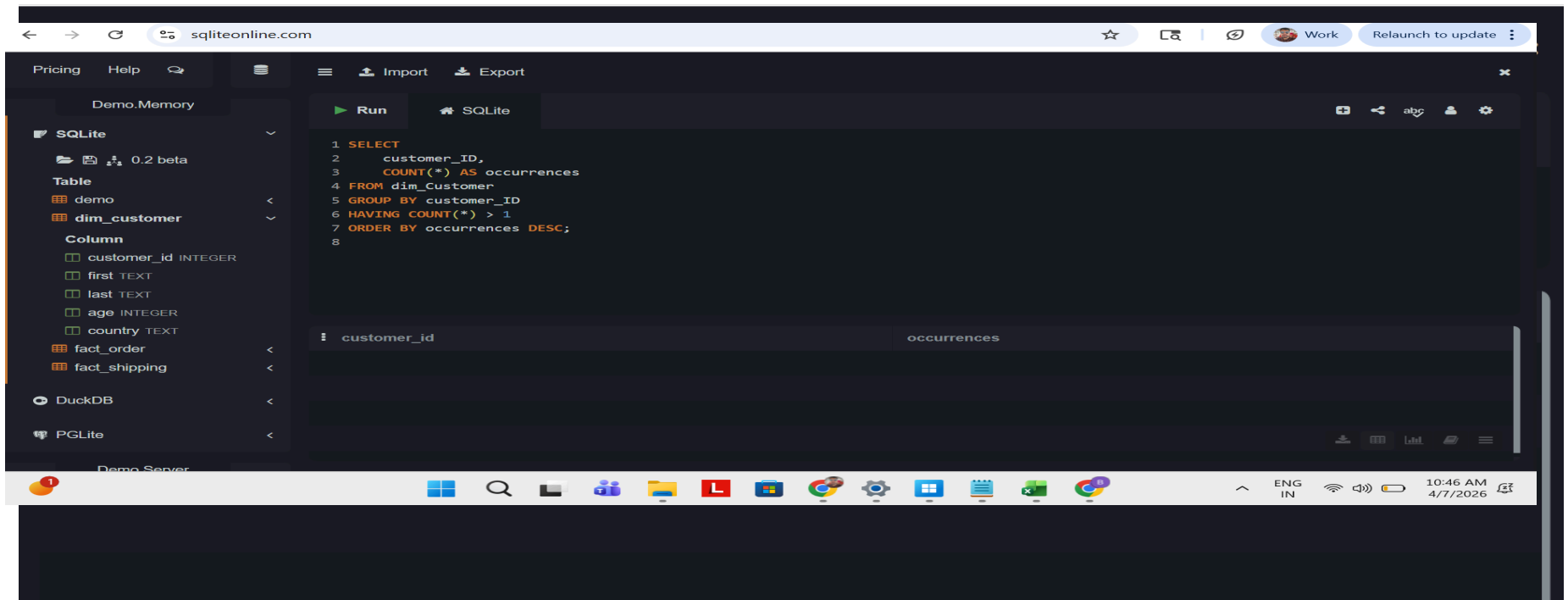
Query : SELECT COUNT(*) AS total_rows,COUNT(*) FILTER (WHERE Shipping_ID IS NULL) AS null_shipping_id, COUNT(*) FILTER (WHERE Status IS NULL) AS null_status, COUNT(*) FILTER (WHERE Customer_ID IS NULL) AS null_customer_id FROM fact_shipping;

2. Accuracy Checks : Customer Table (dim_customer)

Checks for duplicate or identifying primary keys, invalid name formats using regex, age range violations, and country domain compliance.

2.1 Duplicate Customer_ID

Result : 0 duplicates: **customer_id** is a primary key



The screenshot displays the SQLiteOnline.com web application. On the left sidebar, the 'demo' database is selected, and the 'dim_customer' table is highlighted. The table's schema is visible, showing columns: customer_id (INTEGER), first (TEXT), last (TEXT), age (INTEGER), and country (TEXT). The main area shows a SQL query being executed:

```
1 SELECT
2   customer_ID,
3   COUNT(*) AS occurrences
4 FROM dim_Customer
5 GROUP BY customer_ID
6 HAVING COUNT(*) > 1
7 ORDER BY occurrences DESC;
8
```

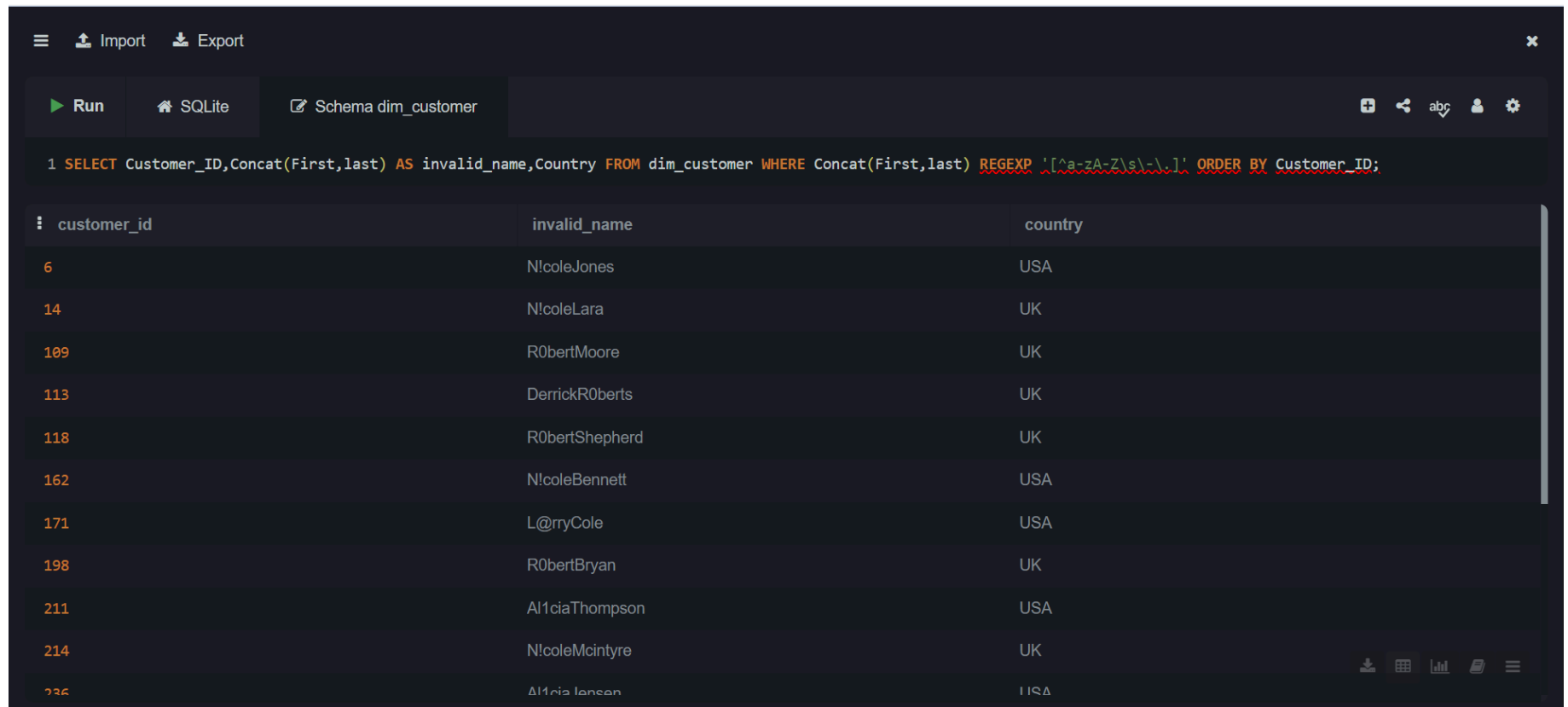
The results table below the query has two columns: 'customer_id' and 'occurrences'. The table is currently empty, indicating no duplicates were found. The bottom of the interface shows a Windows taskbar with various application icons and a system clock indicating 10:46 AM on 4/7/2026.

Query : **SELECT customer_ID, COUNT(*) AS occurrences FROM dim_Customer GROUP BY customer_ID HAVING COUNT(*) > 1 ORDER BY occurrences DESC;**

2.2 Invalid characters in Name (First + Last)

A regex pattern is applied to the combination First name and last name column to detect any character that is not a letter (a-z, A-Z), space, hyphen, or dot. This catches substitutions like N!cole, R0bert, L@rry, and Al1cia.

Result : There are 12 Invalid names that uses Special Characters.



The screenshot shows a SQL IDE interface with a dark theme. At the top, there are buttons for 'Import' and 'Export'. Below that, a 'Run' button is visible. The main area displays a SQL query: `1 SELECT Customer_ID,Concat(First,last) AS invalid_name,Country FROM dim_customer WHERE Concat(First,last) REGEXP '^[^a-zA-Z\s\-\.\.]' ORDER BY Customer_ID;`. Below the query, a table of results is shown with columns 'customer_id', 'invalid_name', and 'country'. The table lists 12 rows of data, each representing a customer with an invalid name. The names include special characters like '!', '0', '@', and '1'.

customer_id	invalid_name	country
6	N!coleJones	USA
14	N!coleLara	UK
109	R0bertMoore	UK
113	DerrickR0berts	UK
118	R0bertShepherd	UK
162	N!coleBennett	USA
171	L@rryCole	USA
198	R0bertBryan	UK
211	Al1ciaThompson	USA
214	N!coleMcintyre	UK
236	Al1cia Jensen	USA

Query : `SELECT Customer_ID,Concat(First,last) AS invalid_name,Country FROM dim_customer WHERE Concat(First,last) REGEXP '^[^a-zA-Z\s\-\.\.]' ORDER BY Customer_ID;`

2.4 Age range check — min, max, avg and out-of-range count

MIN, MAX, and AVG confirm the distribution is reasonable. The FILTER clause counts any ages outside the plausible human range of 1 to 120.

Run

SQLite

```
1 SELECT MIN(Age) AS min_age, MAX(Age) AS max_age, ROUND(AVG(Age), 1) AS avg_age,
2 COUNT(*) FILTER (WHERE Age < 1 OR Age > 120) AS out_of_range_count, COUNT(*) FILTER (WHERE Age < 18) AS under_18_count
3 FROM dim_customer;
```

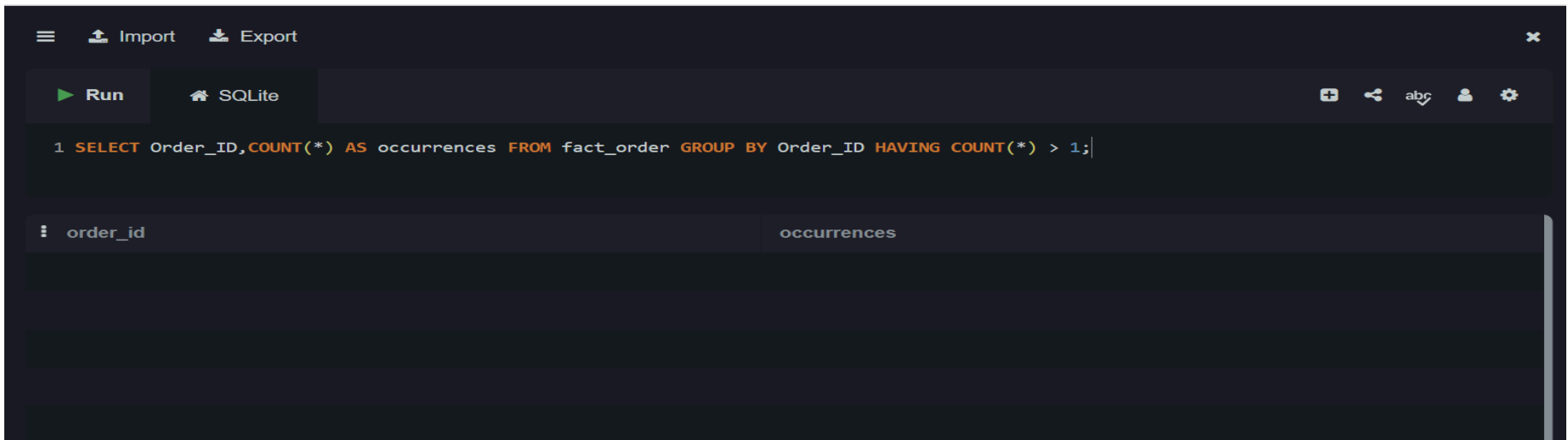
min_age	max_age	avg_age	out_of_range_count	under_18_count
18	80	47.6	0	0

3. Accuracy Checks : fact_order Table

Checks for duplicate order IDs, invalid amount values, product name consistency, and customers with multiple orders.

3.1 Duplicate Order_ID

Result : 0 duplicate Order_id, This Column is primary key in fact_order



The screenshot shows a SQL query editor interface. At the top, there are buttons for 'Import' and 'Export'. Below that, a 'Run' button is visible. The query text is: `1 SELECT Order_ID,COUNT(*) AS occurrences FROM fact_order GROUP BY Order_ID HAVING COUNT(*) > 1;`. Below the query, there is a table with two columns: 'order_id' and 'occurrences'. The table is currently empty, indicating that no duplicate order IDs were found.

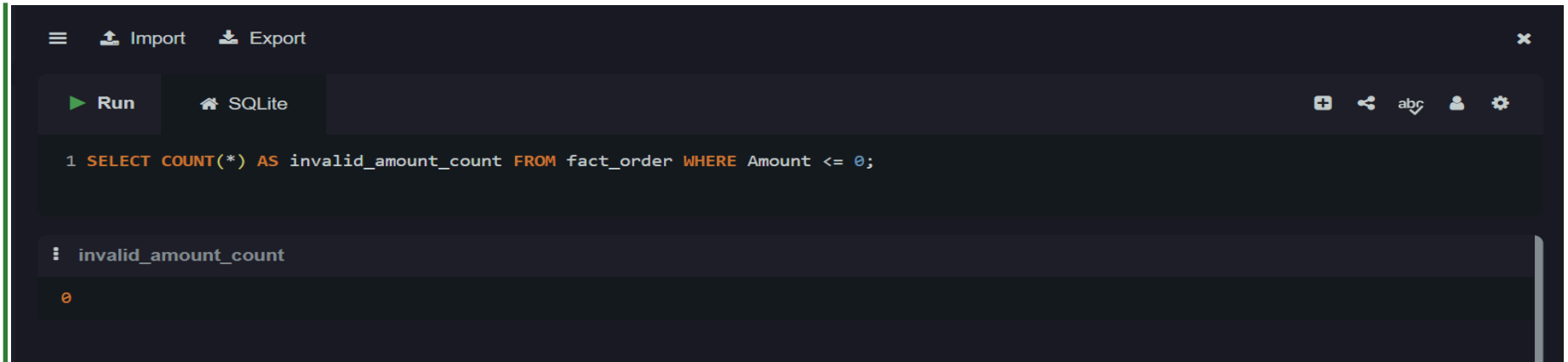
order_id	occurrences
----------	-------------

Query : `SELECT Order_ID,COUNT(*) AS occurrences FROM fact_order GROUP BY Order_ID HAVING COUNT(*) > 1;`

3.2 Negative or zero Amount check

A WHERE Amount <= 0 filter counts any orders where the monetary value is zero or negative, which would indicate a data entry or system error.

Result :No Invalid Amount found



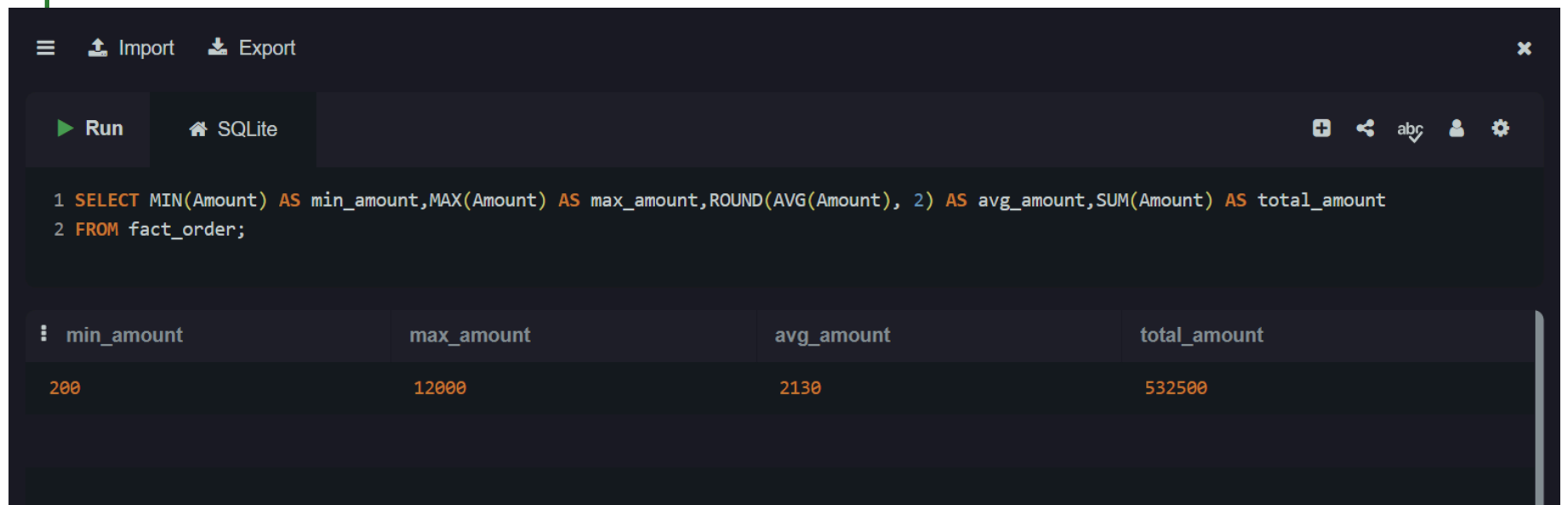
The screenshot shows a SQL query editor interface. At the top, there are buttons for 'Import' and 'Export'. Below that, a 'Run' button is visible. The query is written in a dark-themed editor. The result is displayed in a table below the query.

```
1 SELECT COUNT(*) AS invalid_amount_count FROM fact_order WHERE Amount <= 0;
```

invalid_amount_count
0

3.3 Amount statistics : min, max, avg, total

Aggregate functions confirm the realistic spread of order values across the dataset. **Note: Currency Column is not available , hence cannot have realistic meaning to the amount here.**



The screenshot shows a SQL query editor interface. At the top, there are buttons for 'Import' and 'Export'. Below that, a 'Run' button is visible. The query is written in a dark-themed editor. The result is displayed in a table below the query.

```
1 SELECT MIN(Amount) AS min_amount, MAX(Amount) AS max_amount, ROUND(AVG(Amount), 2) AS avg_amount, SUM(Amount) AS total_amount  
2 FROM fact_order;
```

min_amount	max_amount	avg_amount	total_amount
200	12000	2130	532500

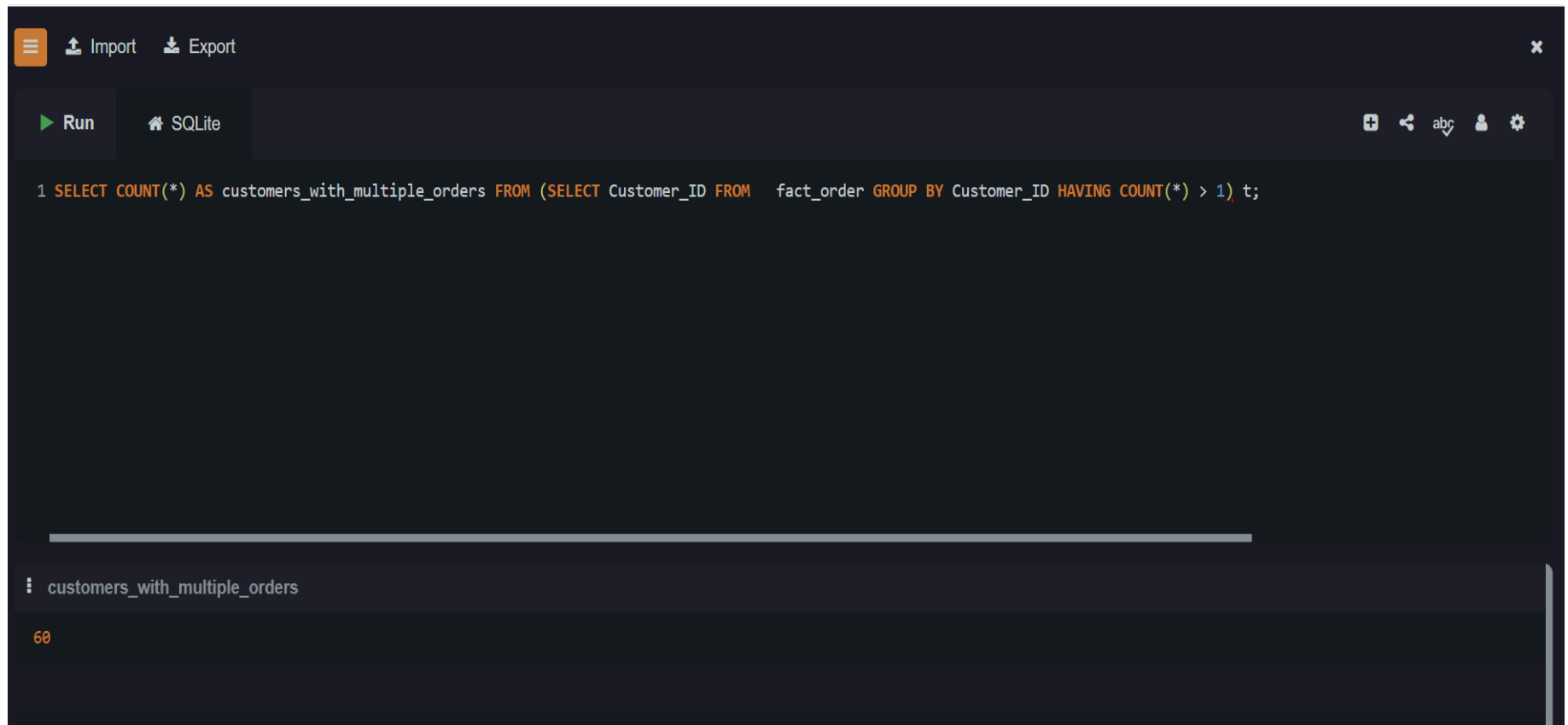
3.5 Customers with multiple orders

Query groups orders by Customer_ID and counts how many customers have placed more than one order.

Result : There are 60 Customers with multiple orders, Customer_id column is the Foreign key here

Note: *Since there are Multiple order per customer and Multiple shipments per customer, and no data to link particular order to particular shipment, This creates a huge data gap.*

This is the root cause of the many-to-many risk when joining to fact_shipping via Customer_ID. A customer with 3 orders and 2 shipments produces 6 joined rows instead of 3.



The screenshot shows a SQL query editor interface. At the top, there are buttons for 'Import' and 'Export'. Below that, a 'Run' button and a 'SQLite' database selector are visible. The main area contains a SQL query: `1 SELECT COUNT(*) AS customers_with_multiple_orders FROM (SELECT Customer_ID FROM fact_order GROUP BY Customer_ID HAVING COUNT(*) > 1) t;`. At the bottom, the results are displayed in a table with one row and one column, showing the value '60'.

```
1 SELECT COUNT(*) AS customers_with_multiple_orders FROM (SELECT Customer_ID FROM fact_order GROUP BY Customer_ID HAVING COUNT(*) > 1) t;
```

customers_with_multiple_orders
60

4. Accuracy Checks : fact_shipping Table

Checks for duplicate shipping IDs, status domain validation, and customers with multiple shipment records.

4.1 Duplicate Shipping_ID

Result : 0 duplicate shipping_id, This Shipping_id is primary key in fact_shipping



The screenshot shows a SQL query editor interface. At the top, there are buttons for 'Import' and 'Export'. Below that, a 'Run' button and a 'SQLite' database selector are visible. The query area contains the following SQL statement:

```
1 SELECT Shipping_ID,COUNT(*) AS occurrences FROM fact_shipping GROUP BY Shipping_ID HAVING COUNT(*) > 1
```

Below the query, a table structure is shown with two columns: 'shipping_id' and 'occurrences'. The table is currently empty, indicating that no duplicate shipping IDs were found.

shipping_id	occurrences
-------------	-------------

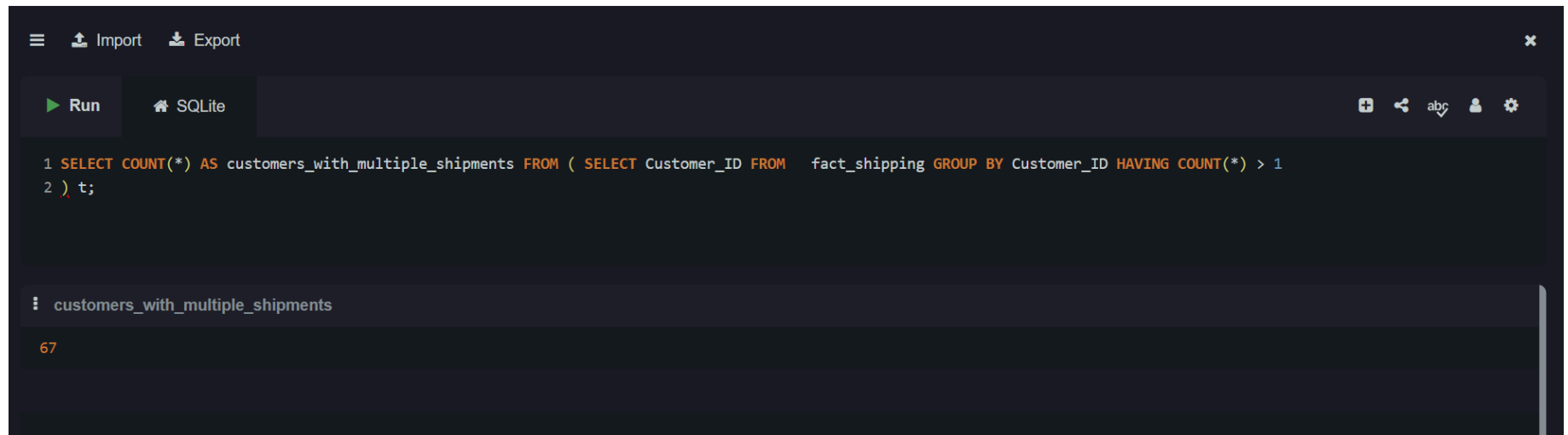
4.2 Customers with multiple shipments

Query groups shipment by Customer_ID and counts how many customers has placed more than one shipment.

Result : There are 67 Customers with multiple shipment, Customer_id column is the Foreign key here

Note: *Since there are Multiple order per customer and Multiple shipments per customer, and no data to link particular order to particular shipment, This creates a huge data gap.*

This is the root cause of the many-to-many risk when joining to fact_shipping via Customer_ID. A customer with 3 orders and 2 shipments produces 6 joined rows instead of 3.



The screenshot shows a SQL query editor interface with a dark theme. At the top, there are icons for 'Import' and 'Export'. Below that, a 'Run' button and a 'SQLite' database selector are visible. The main area contains a SQL query:

```
1 SELECT COUNT(*) AS customers_with_multiple_shipments FROM ( SELECT Customer_ID FROM fact_shipping GROUP BY Customer_ID HAVING COUNT(*) > 1
2 ) t;
```

 Below the query, the results are displayed in a table with one column named 'customers_with_multiple_shipments' and one row containing the value '67'.

```
1 SELECT COUNT(*) AS customers_with_multiple_shipments FROM ( SELECT Customer_ID FROM fact_shipping GROUP BY Customer_ID HAVING COUNT(*) > 1
2 ) t;
```

customers_with_multiple_shipments
67

5. Referential Integrity Checks

Validates FK relationships across all three tables and identifies customers not represented in each fact table.

5.1 Orphan orders : Customer_ID in fact_order but not found in Customer

Result : There are No order rows in Fact_order table whose Customer_ID has no matching row in the Customer table. No missing Customer details in Customer_table

☰ ⬆ Import ⬇ Export

▶ Run

🏠 SQLite

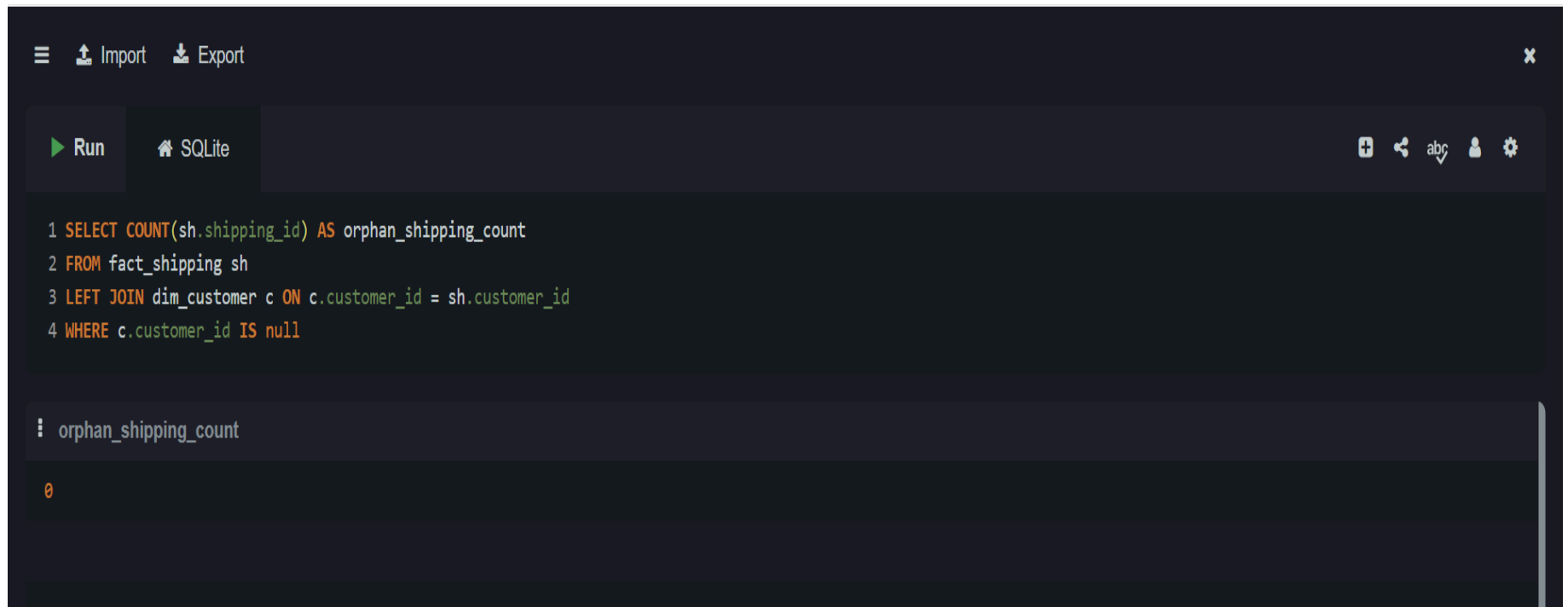
```
1 SELECT COUNT(fo.order_id) AS orphan_order_count
2 FROM fact_order fo
3 LEFT JOIN dim_customer c ON c.customer_id = fo.customer_id
4 WHERE c.customer_id IS null
```

⋮ orphan_order_count

0

5.2 Orphan shipments : Customer_ID in fact_shipping not found in Customer

Result : There are No rows in Fact_shipping table whose Customer_ID has no matching row in the Customer table. No missing Customer details in Customer_table related to shipment



The screenshot shows a SQL query execution interface with a dark theme. At the top, there are icons for 'Import' and 'Export'. Below that, there are buttons for 'Run' and 'SQLite'. The SQL query is displayed in a text area, and the result is shown in a table below it.

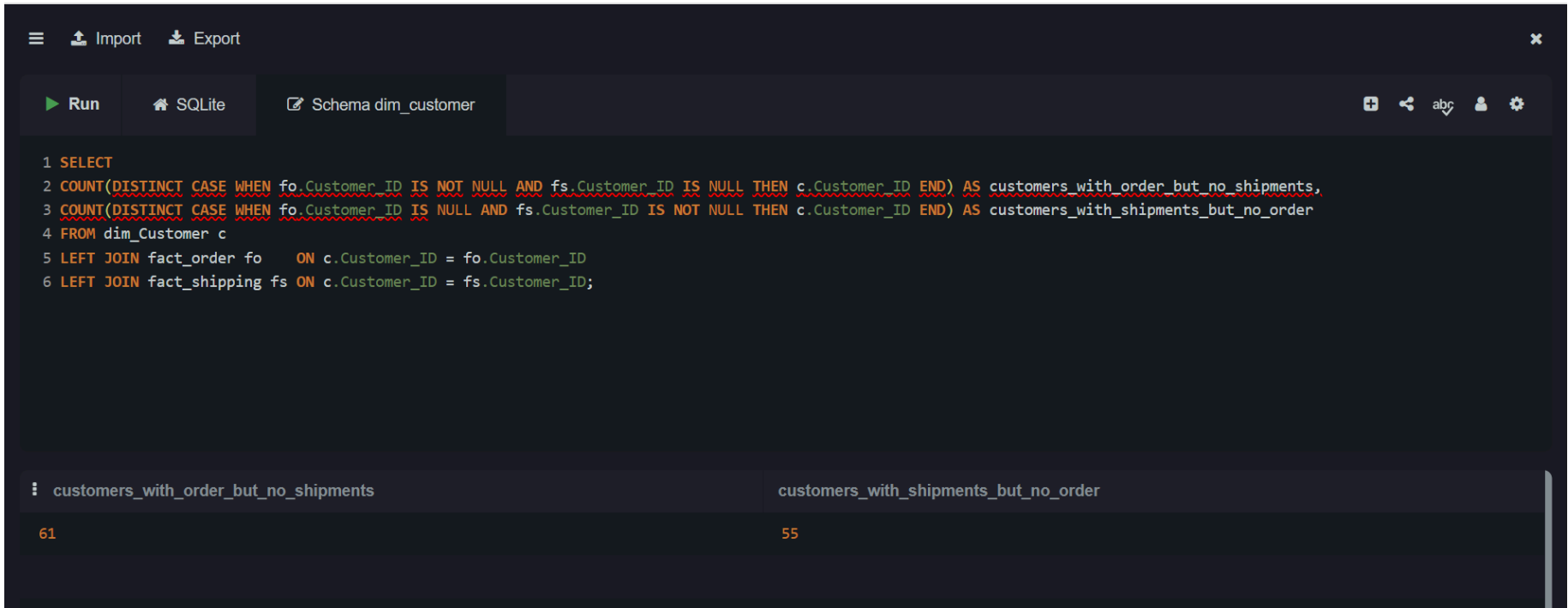
```
1 SELECT COUNT(sh.shipping_id) AS orphan_shipping_count
2 FROM fact_shipping sh
3 LEFT JOIN dim_customer c ON c.customer_id = sh.customer_id
4 WHERE c.customer_id IS null
```

orphan_shipping_count
0

5.3 Customers Engagement with their Ordering and Shipment details

Key Observation :

1. There are 61 customers who have placed orders that have no corresponding shipment records. - Major gap in shipping data
2. The data indicates there are 55 customers with shipment records but no order records. - we shouldn't be shipping items that haven't been ordered. Hence Big gap with data in Shipping and order.



The screenshot shows a SQL query editor interface with a dark theme. At the top, there are buttons for 'Run', 'SQLite', and 'Schema dim_customer'. The query is as follows:

```
1 SELECT
2 COUNT(DISTINCT CASE WHEN fo.Customer_ID IS NOT NULL AND fs.Customer_ID IS NULL THEN c.Customer_ID END) AS customers_with_order_but_no_shipments,
3 COUNT(DISTINCT CASE WHEN fo.Customer_ID IS NULL AND fs.Customer_ID IS NOT NULL THEN c.Customer_ID END) AS customers_with_shipments_but_no_order
4 FROM dim_Customer c
5 LEFT JOIN fact_order fo ON c.Customer_ID = fo.Customer_ID
6 LEFT JOIN fact_shipping fs ON c.Customer_ID = fs.Customer_ID;
```

Below the query, the results are displayed in a table with two columns: 'customers_with_order_but_no_shipments' and 'customers_with_shipments_but_no_order'. The values are 61 and 55 respectively.

customers_with_order_but_no_shipments	customers_with_shipments_but_no_order
61	55

```
SELECT
COUNT(DISTINCT CASE WHEN fo.Customer_ID IS NOT NULL AND fs.Customer_ID IS NULL THEN c.Customer_ID END) AS customers_with_order_but_no_shipments,
COUNT(DISTINCT CASE WHEN fo.Customer_ID IS NULL AND fs.Customer_ID IS not NULL THEN c.Customer_ID END) AS customers_with_shipments_but_no_order
FROM dim_Customer c
LEFT JOIN fact_order fo ON c.Customer_ID = fo.Customer_ID
LEFT JOIN fact_shipping fs ON c.Customer_ID = fs.Customer_ID;
```